



Local Search & Problem Reduction

Hill Climbing · Simulated Annealing · AND–OR Graphs · AO*

Course No: 0613-401

Course Title: Artificial Intelligence

Ekramul Haque Amin

Lecturer

Department of Artificial Intelligence and Data Science

Dhaka International University (DIU)

Today's Content

- ▶ **Recap: informed search (Greedy, A*)**
- ▶ **Local search and the state-space landscape**
- ▶ **Hill Climbing — step by step, 8-queens, Python**
- ▶ **Escaping local maxima: random restart, simulated annealing**
- ▶ **Problem reduction and AND–OR graphs**
- ▶ **The AO* algorithm — step by step**
- ▶ **Class exercise**

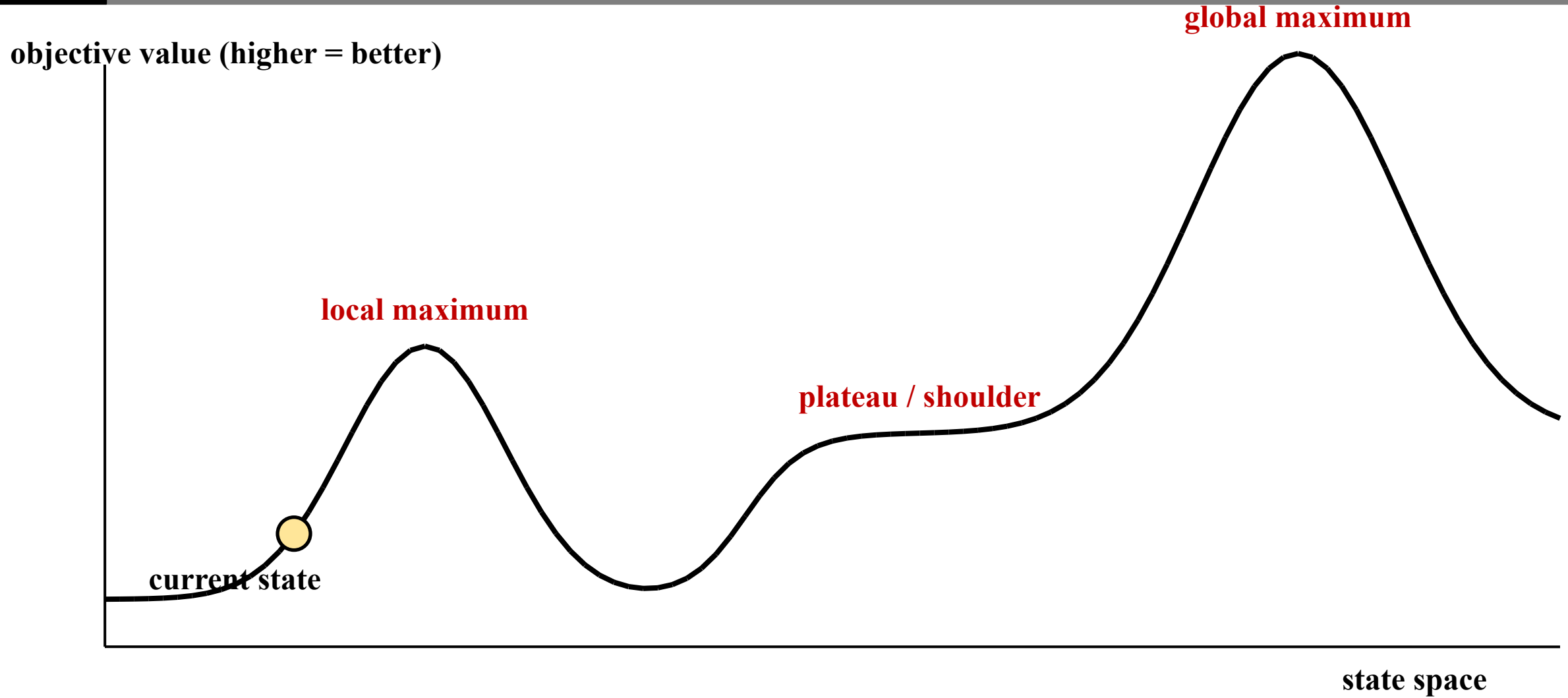
Recap: Informed Search

- ▶ A heuristic $h(n)$ estimates the cost from n to the goal (e.g. straight-line distance).
- ▶ Greedy Best-First: expand lowest h — fast, but returned cost 11 instead of 10 on the trap graph.
- ▶ A*: expand lowest $f = g + h$ — found the optimal 275 km Dhaka → Chattogram route with 6 expansions.
- ▶ A* is optimal when h is admissible (never overestimates).
- ▶ All these keep a whole frontier in memory and return a path.
- ▶ Today: algorithms that keep only one state (local search), and problems whose solution is not a single path (AND–OR graphs).

Local Search: A Different Idea

- ▶ Sometimes we only need a good final state, not the path: 8-queens, class timetables, delivery routes.
- ▶ Local search keeps only one current state and moves to a neighbouring state.
- ▶ Uses very little memory — no frontier, no explored set.
- ▶ Works on huge or infinite state spaces.
- ▶ Not systematic: it can miss the best solution.
- ▶ The most basic local search: Hill Climbing.

The State-Space Landscape



Hill Climbing Algorithm

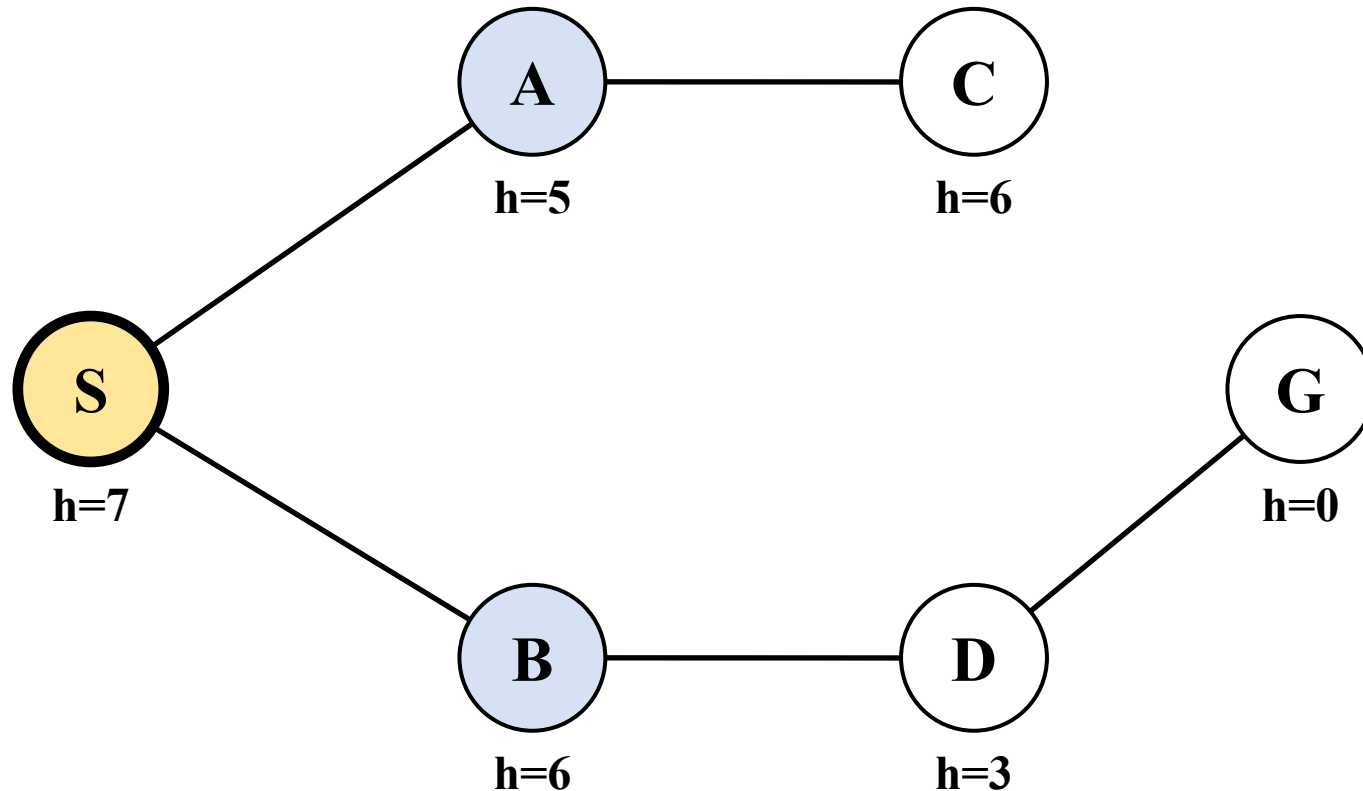
```
function HILL-CLIMBING(problem) returns a state
  current ← initial state
  loop do
    neighbour ← best successor of current
    if value(neighbour) ≤ value(current) then
      return current          // a peak (maybe local!)
    current ← neighbour
```

- ▶ Looks only at immediate neighbours — “climbing a hill in fog”.
- ▶ Never goes downhill, never backtracks.
- ▶ With a cost h (lower is better), move to the neighbour with the lowest h .

Hill Climbing: Step by Step (1/5)

Step 1: at S ($h = 7$)

Neighbours: A ($h = 5$), B ($h = 6$).
Best is A, and $5 < 7 \rightarrow$ move to A.



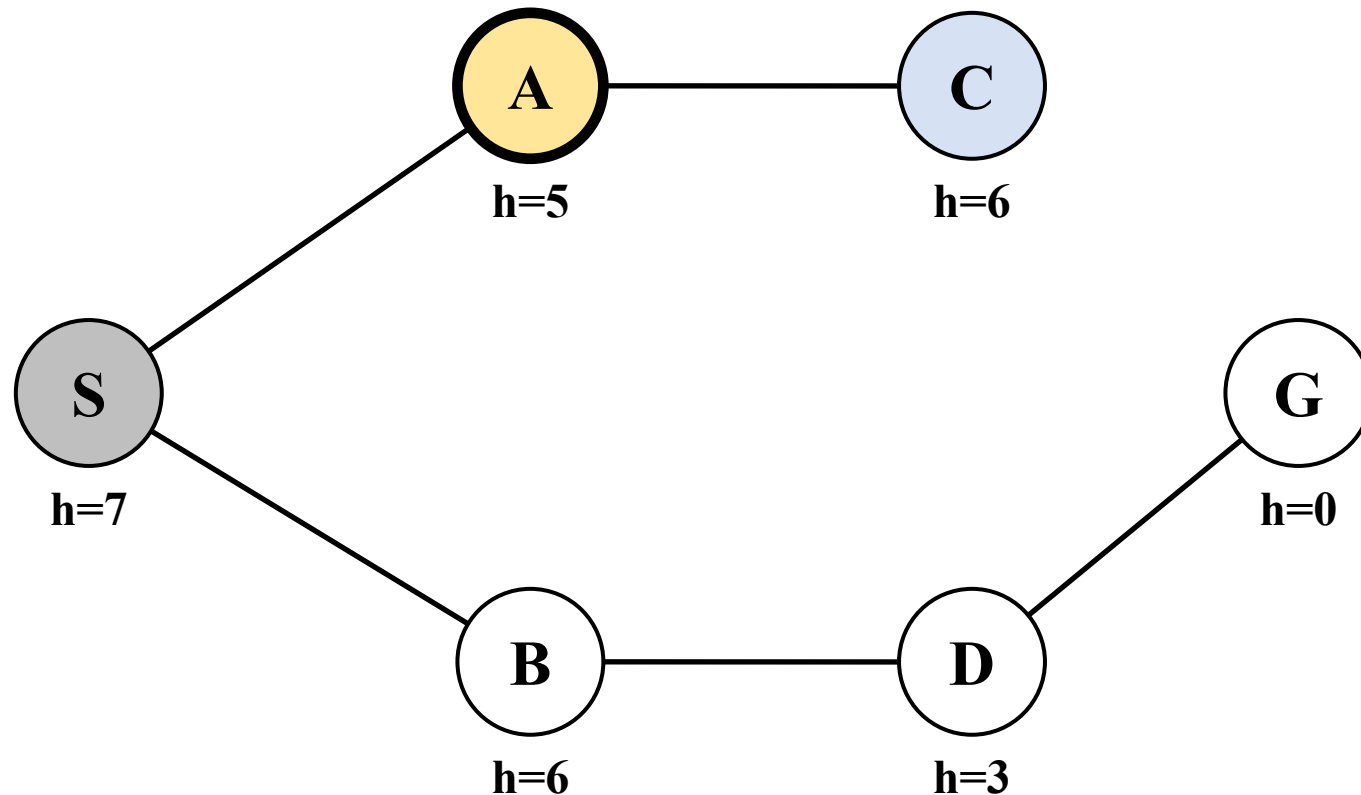
 Current

 Neighbour

 Visited

Lower h is better here (we walk downhill toward $h = 0$).

Hill Climbing: Step by Step (2/5)



Current

Neighbour

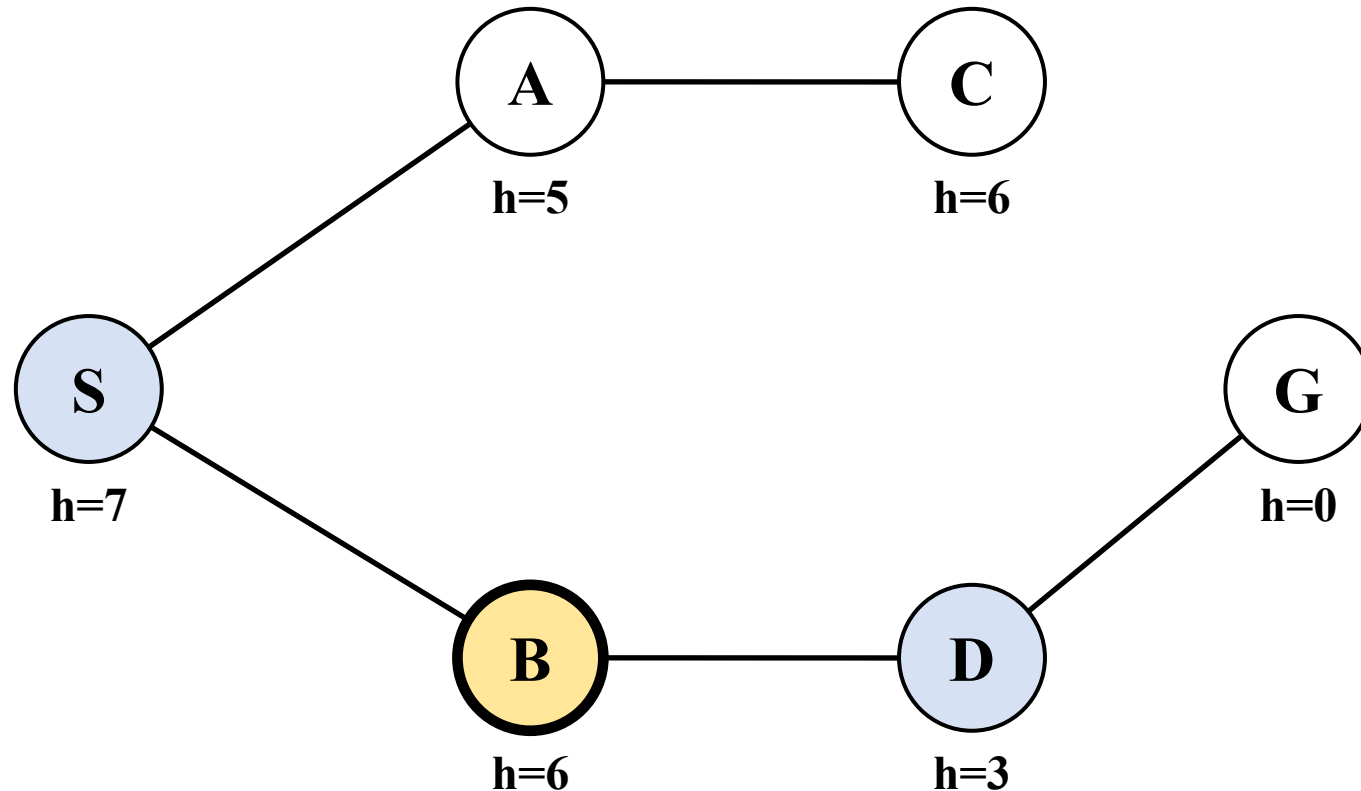
Visited

Step 2: at A ($h = 5$)

Neighbours: S ($h = 7$), C ($h = 6$).
Both are **WORSE** than 5 $\rightarrow$ stop.
Stuck at A: a local minimum! The goal G is never reached.

Lower h is better here (we walk downhill toward $h = 0$).

Hill Climbing: Step by Step (3/5)



Random restart at B ($h = 6$)

Neighbours: S (7), D (3). Move to D.

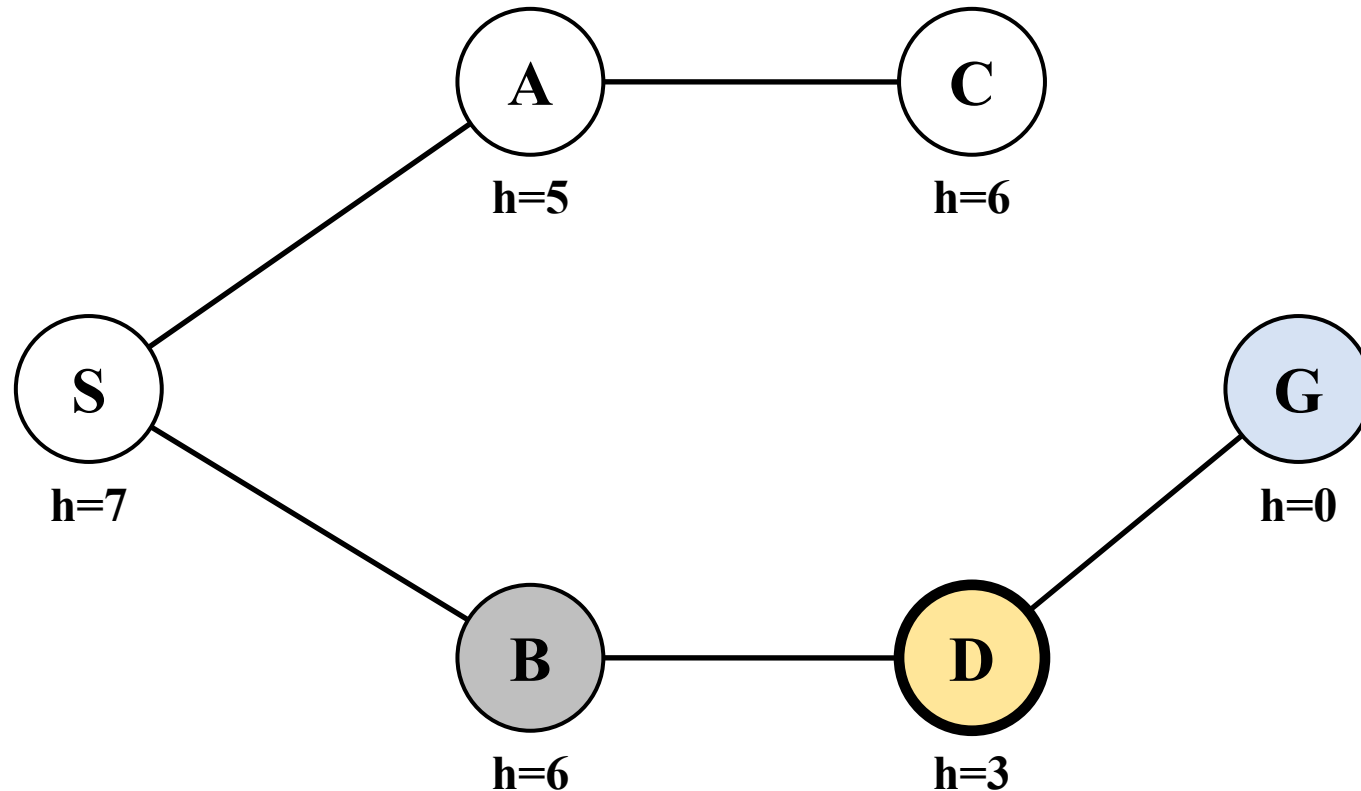
Lower h is better here (we walk downhill toward $h = 0$).

Current

Neighbour

Visited

Hill Climbing: Step by Step (4/5)



At D ($h = 3$)

Neighbours: B (6), G (0). Move to G.

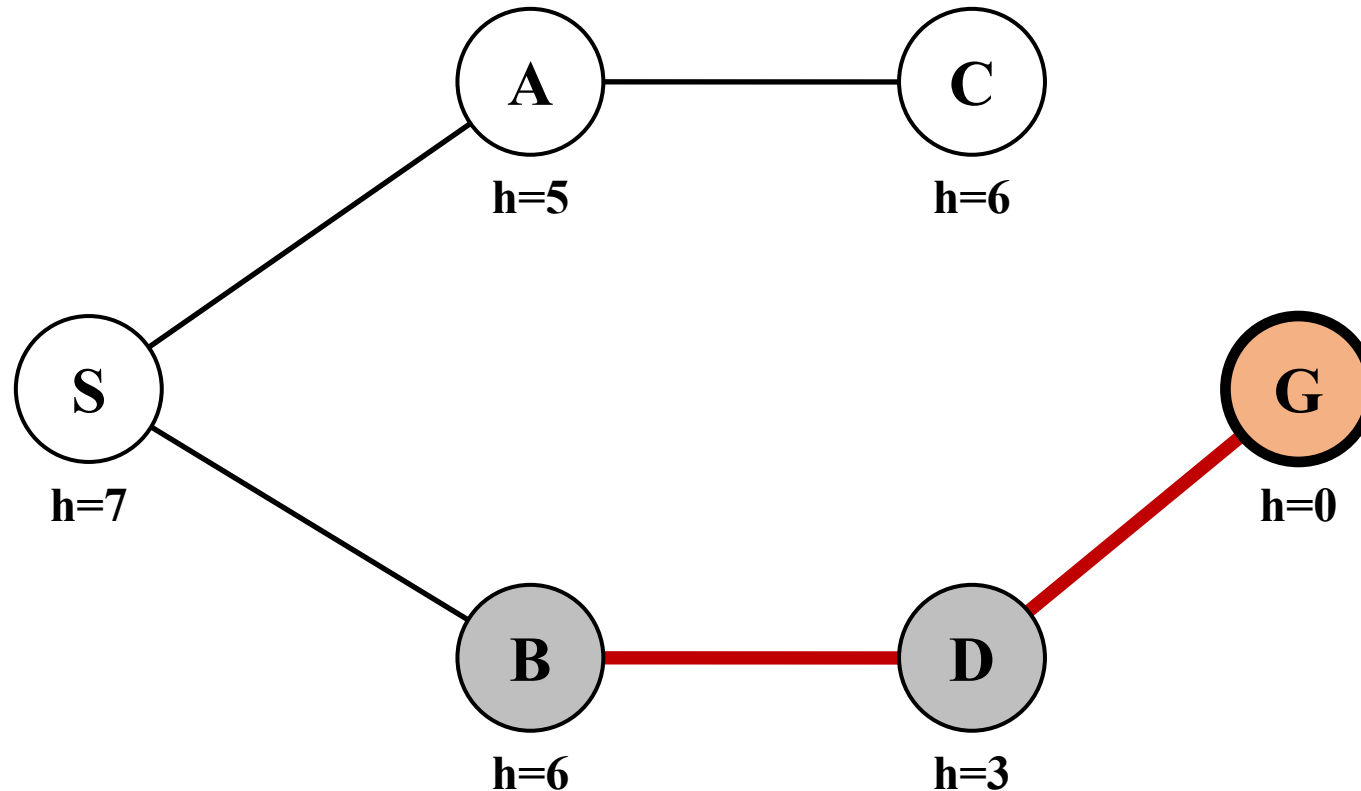
Lower h is better here (we walk downhill toward $h = 0$).

 **Current**

 **Neighbour**

 **Visited**

Hill Climbing: Step by Step (5/5)



At G ($h = 0$) — GOAL ✓

**Restarting from another state
escaped the local minimum: $B \rightarrow D \rightarrow G$.**

**Lower h is better here (we walk downhill
toward $h = 0$).**

 **Current**

 **Neighbour**

 **Visited**

Example: 8-Queens with Hill Climbing

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	Q	13	16	13	16
Q	14	17	15	Q	14	16	16
17	Q	16	18	15	Q	15	Q
18	14	Q	15	15	14	Q	16
14	14	13	17	12	14	12	18

- ▶ Place 8 queens so that no two attack each other.
- ▶ State: one queen in each column.
- ▶ Neighbour: move one queen within its column ($8 \times 7 = 56$ neighbours).
- ▶ h = number of attacking pairs. Current board: $h = 17$
- ▶ Numbers = h after that move.
- ▶ Yellow squares = best moves ($h = 12$). Hill climbing picks one.

8-Queens: After One Step

	Q						
			Q				
Q				Q			
					Q		Q
		Q				Q	

- ▶ Queen in column 2 moved to the top row → $h = 12$
- ▶ Repeat until no move lowers h .
- ▶ Result on random 8-queens boards:
 - solved about 14% of the time
 - stuck about 86% of the time (local minimum)
- ▶ So we need ways to escape...

Why Hill Climbing Gets Stuck

Problem	What happens	Picture
Local maximum	A peak higher than its neighbours but lower than the best peak	Small hill next to a mountain
Plateau	A flat area: all neighbours have the same value → no direction	Flat field in fog
Ridge	A sequence of local maxima that single moves cannot climb	Narrow mountain ridge

- ▶ Hill climbing is not complete and not optimal.
- ▶ But it is fast and uses almost no memory.

Escaping: Variants of Hill Climbing

Variant	Idea
Random-restart	Run hill climbing many times from random starts; keep the best result
Sideways moves	Allow moves to equal-valued neighbours to cross a plateau (limit the number)
Stochastic	Choose randomly among the uphill moves
Simulated annealing	Sometimes accept a WORSE move, with probability $e^{\Delta E/T}$; T (temperature) slowly decreases

Temperature T	10	1	0.1
P(accept a move that is 2 worse)	0.82	0.14	≈ 0

Hill Climbing in Python (Lab Link)

```
def hill_climbing(start, neighbours, h):
    current = start
    while True:
        best = min(neighbours(current), key=h)
        if h(best) >= h(current):    # no better neighbour
            return current
        current = best

graph = {'S': ['A', 'B'], 'A': ['S', 'C'], 'B': ['S', 'D'],
        'C': ['A'], 'D': ['B', 'G'], 'G': ['D']}
h = {'S': 7, 'A': 5, 'B': 6, 'C': 6, 'D': 3, 'G': 0}

print(hill_climbing('S', graph.get, h.get))    # A    (stuck!)
print(hill_climbing('B', graph.get, h.get))    # G    (goal)
```

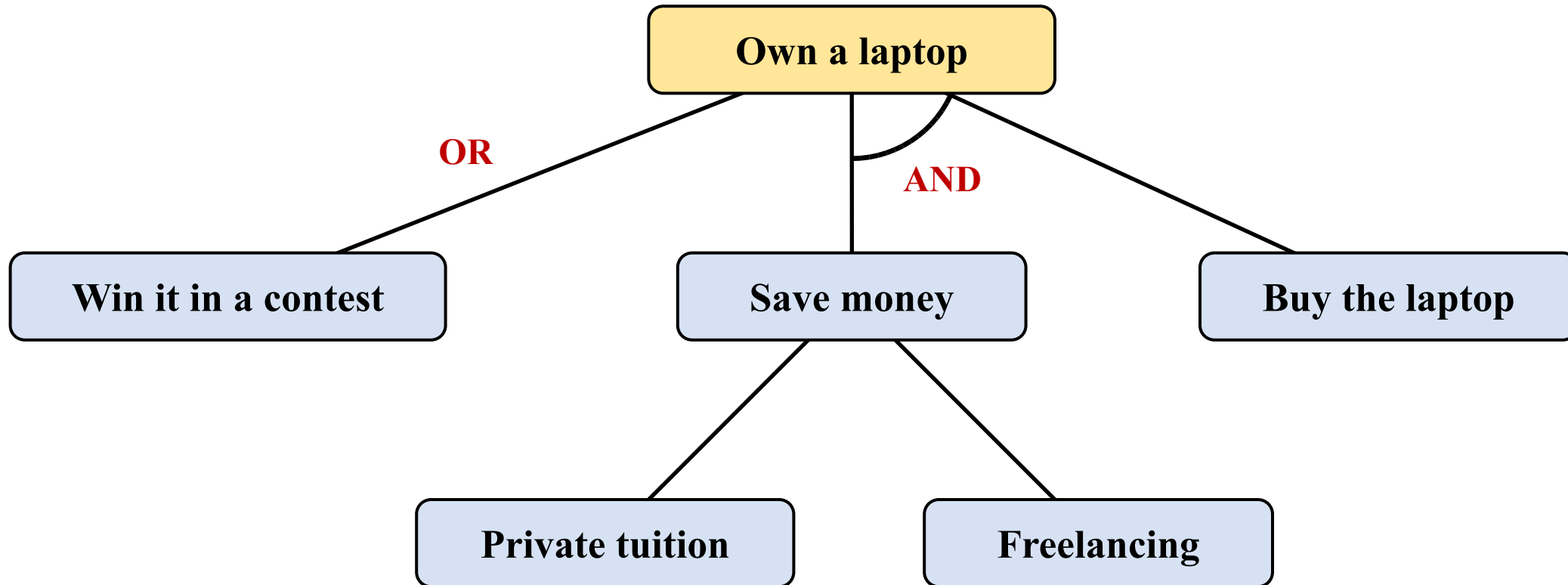
A* vs Hill Climbing

	A* (systematic)	Hill Climbing (local)
Keeps in memory	Frontier + explored set	Only the current state
Returns	A path to the goal	A good final state
Optimal?	Yes (admissible h)	No
Complete?	Yes	No — can get stuck
Memory	Large (exponential)	Very small (constant)
Good for	Route finding, puzzles	8-queens, scheduling, optimisation

Problem Reduction

- ▶ Some problems are easier to solve by breaking them into smaller sub-problems.
- ▶ Each sub-problem may again be broken down, until we reach primitive (directly solvable) problems.
- ▶ Two kinds of decomposition:
 - OR: several alternatives — solving any one is enough
 - AND: several parts — all must be solved
- ▶ Such problems are represented by an AND–OR graph.
- ▶ Examples: planning a trip, proving a theorem, symbolic integration, getting a degree.

AND-OR Graph: An Example

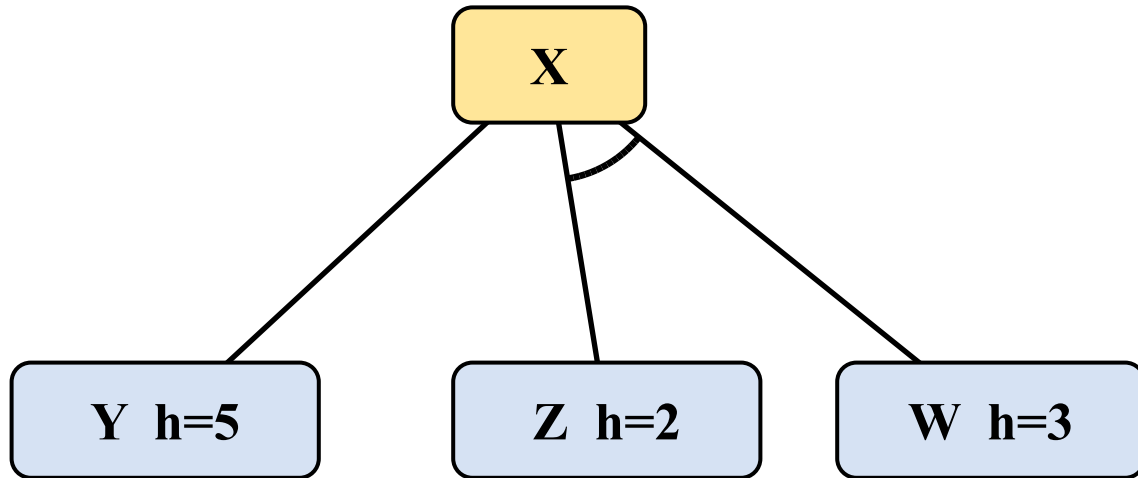


- ▶ The arc joining edges = AND (all children needed). Edges without an arc = OR (choose one).
- ▶ A solution is a sub-graph (not a single path): e.g. Own a laptop ← Save money (by tuition) AND Buy the laptop.

Why Not Just Use A*?

- ▶ A* finds a single path from start to goal.
- ▶ In an AND–OR graph a solution is a solution graph that may contain several branches (all parts of an AND).
- ▶ The cost of an AND choice is the sum of the costs of all its parts.
- ▶ We need an algorithm that:
 - keeps track of the best partial solution graph
 - revises costs upward when a node turns out cheaper or more expensive
 - labels nodes SOLVED when all their needed parts are solved
- ▶ That algorithm is AO*.

Computing Costs in an AND–OR Graph



- ▶ Assume every edge costs 1.
- ▶ OR choice: $\text{cost} = 1 + h(\text{child})$
- ▶ AND choice: $\text{cost} = \Sigma (1 + h(\text{child}))$
- ▶ $f(\text{node}) = \underline{\text{minimum}}$ over its choices

Choice for X	Calculation	Cost
Y (OR)	$1 + 5$	6
Z AND W	$(1 + 2) + (1 + 3)$	7

Best: Y, so $f(X) = 6$

The AO* Algorithm

- 1. Start with the root node only.**
- 2. Follow the marked best arcs from the root → the current best partial solution graph.**
- 3. Pick an unexpanded node on it and expand it. Give each new child its h ; mark primitive children SOLVED.**
- 4. Revise costs upward to the root:**
OR node: $f = \min (1 + f(\text{child}))$; AND choice: $f = \Sigma (1 + f(\text{child}))$
Mark the best choice. If all its children are SOLVED → mark the node SOLVED.
- 5. Repeat until the root is SOLVED (success) or its cost > FUTILITY (fail).**

► Like A*, AO* is guided by $\underline{h}$; with an admissible h it finds the cheapest solution graph.

AO* Search: Step by Step (0/5)



Start

- ▶ Only the root A is known.
- ▶ Each edge costs 1.
- ▶ Number under a node = its current cost estimate f.



Expanding now



Generated

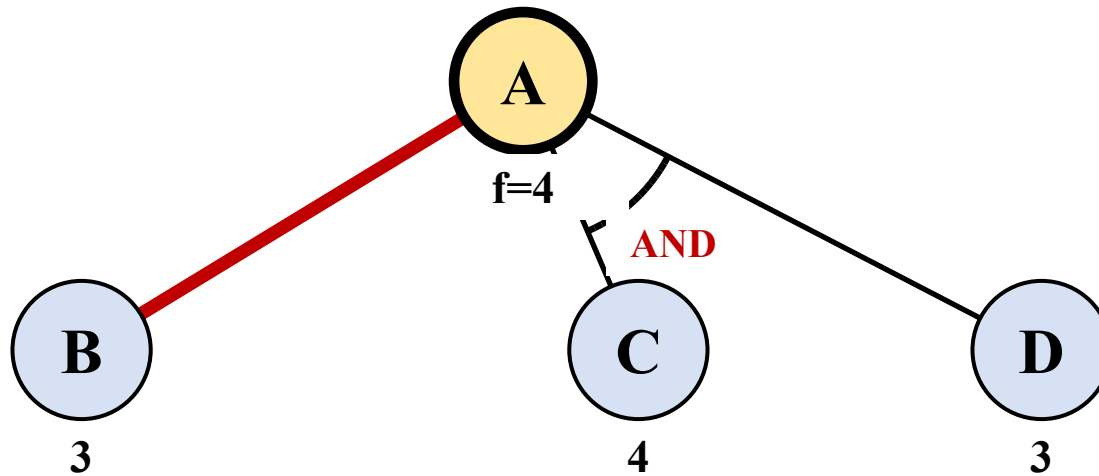


SOLVED ✓



best solution so far

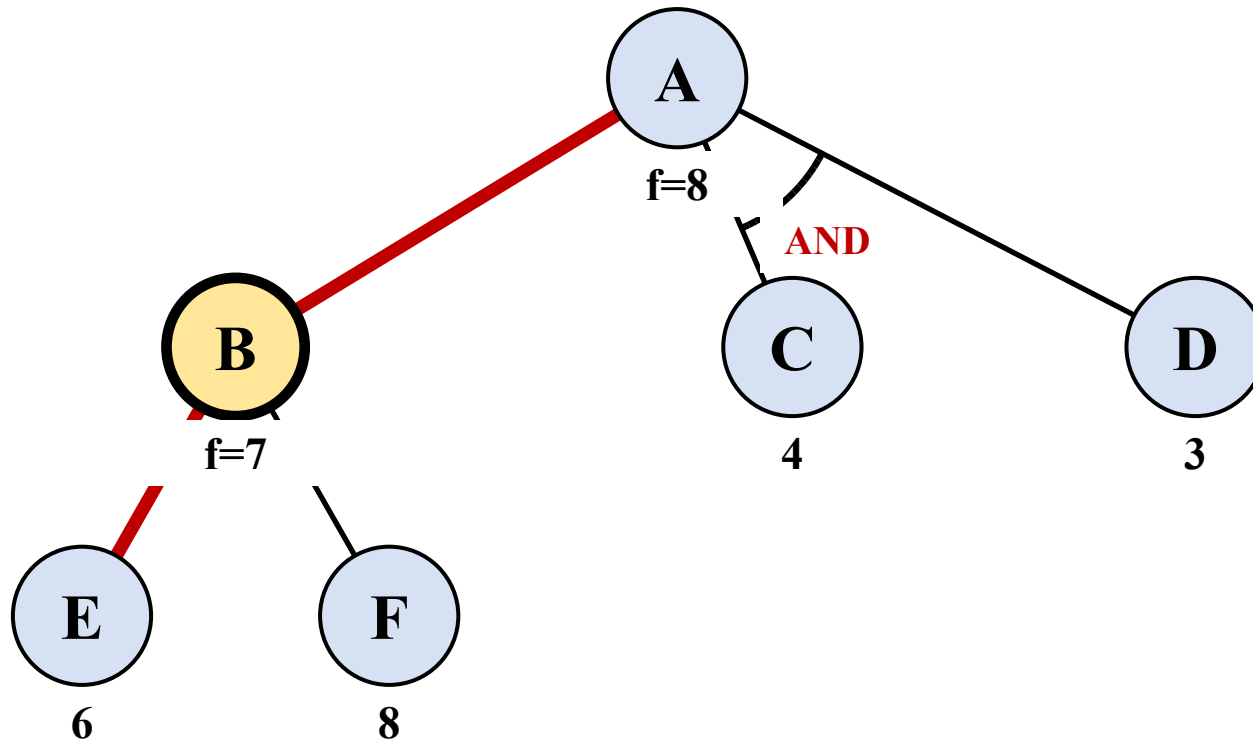
AO* Search: Step by Step (1/5)

**Step 1: expand A**

- ▶ A can be solved by B OR by (C AND D).
- ▶ via B: $1 + 3 = 4$
- ▶ via C AND D: $(1+4) + (1+3) = 9$
- ▶ Best = B $\rightarrow f(A) = 4$

 Expanding now  Generated  SOLVED ✓  best solution so far

AO* Search: Step by Step (2/5)

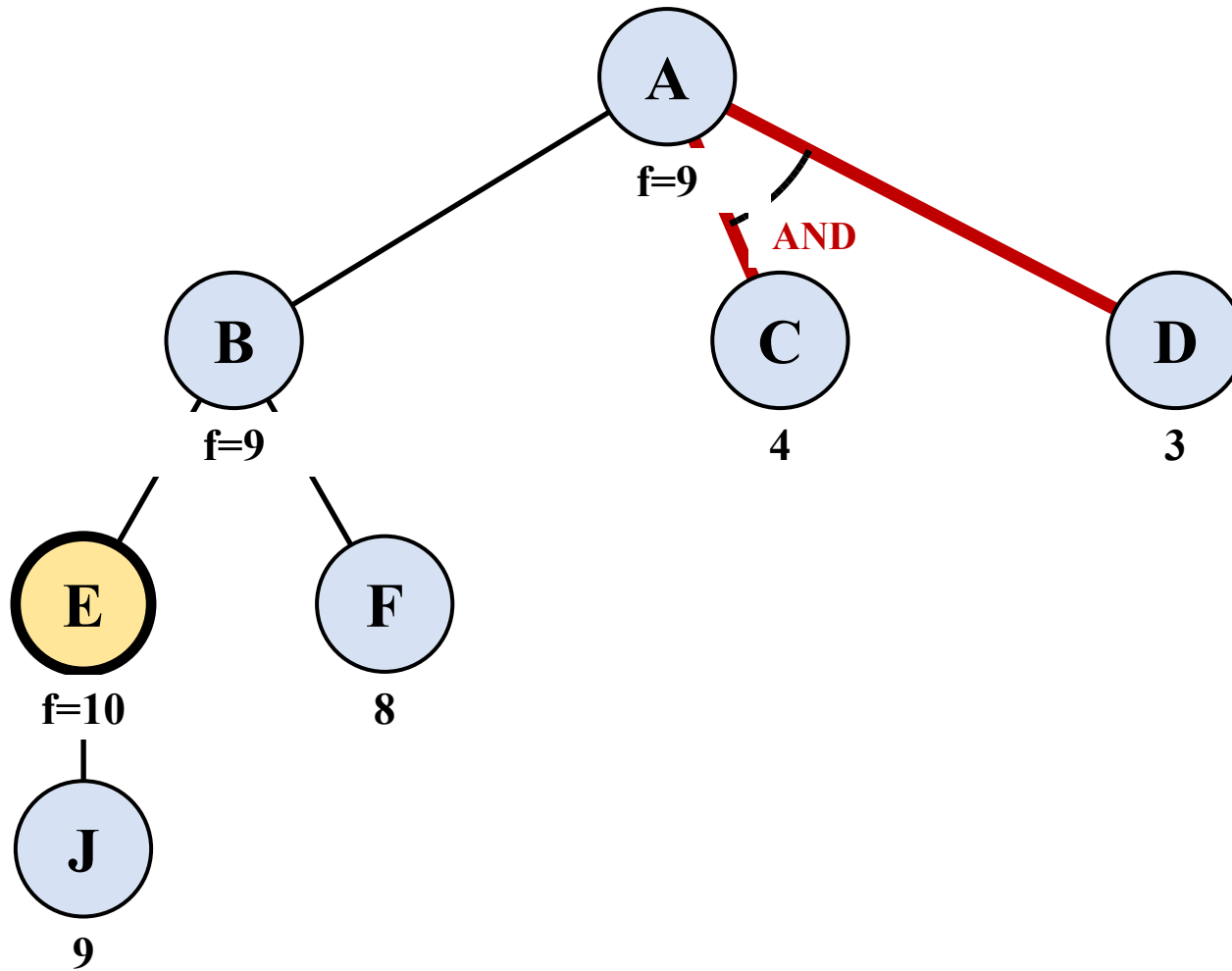


Step 2: expand B

- ▶ $B \rightarrow E \text{ OR } F$
- ▶ $f(B) = \min(1+6, 1+8) = 7$ (via E)
- ▶ Revise A: via B = $1+7 = 8$, via C&D = 9
- ▶ Still $B \rightarrow f(A) = 8$

Expanding now
 Generated
 SOLVED ✓
 best solution so far

AO* Search: Step by Step (3/5)

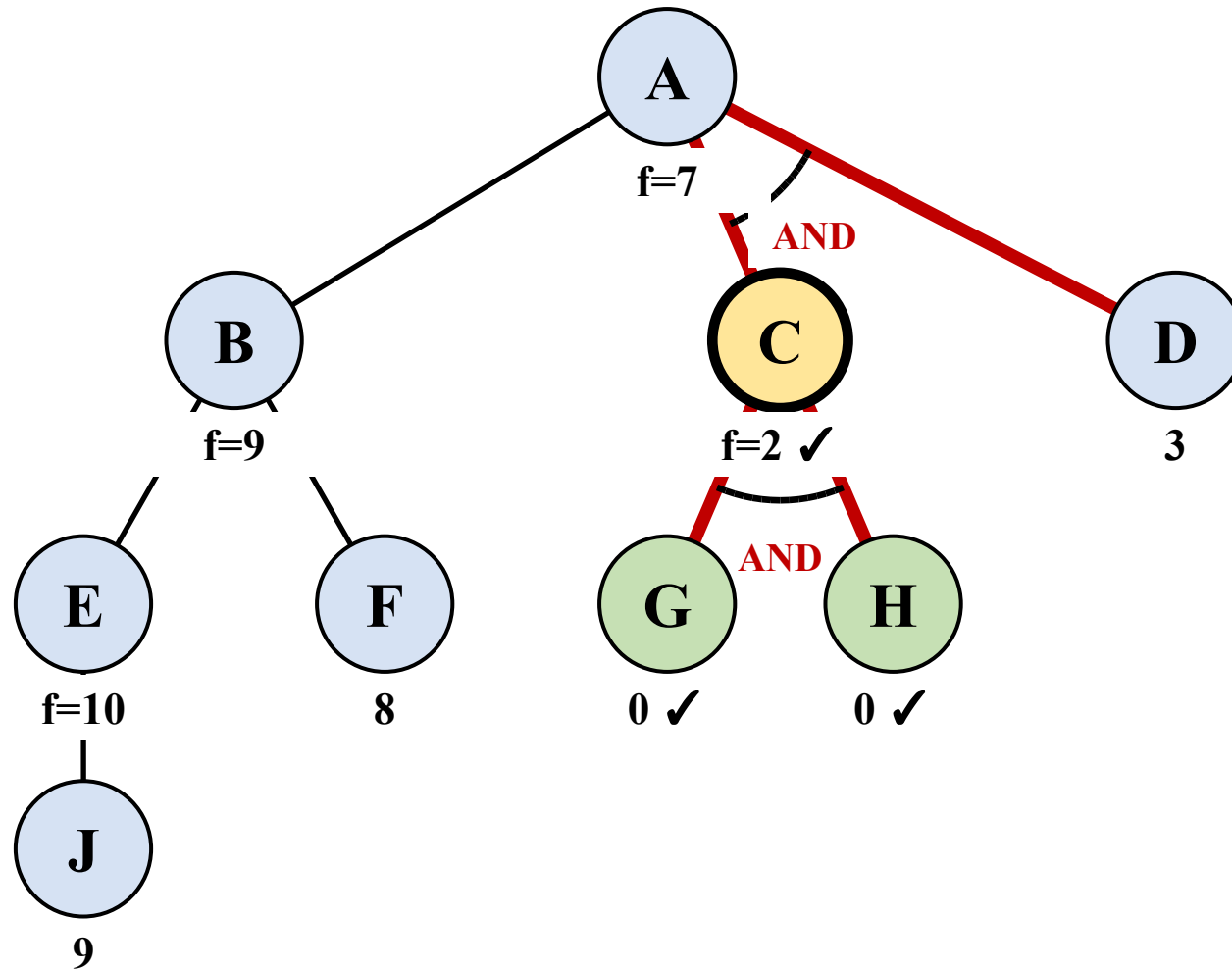


Step 3: expand E

- ▶ **E → J: $f(E) = 1 + 9 = 10$**
- ▶ **$f(B) = \min(1+10, 1+8) = 9$ (now via F)**
- ▶ **Revise A: via B = $1+9 = 10$, via C&D = 9**
- ▶ **SWITCH to C AND D → $f(A) = 9$**

 Expanding now  Generated  SOLVED ✓  best solution so far

AO* Search: Step by Step (4/5)

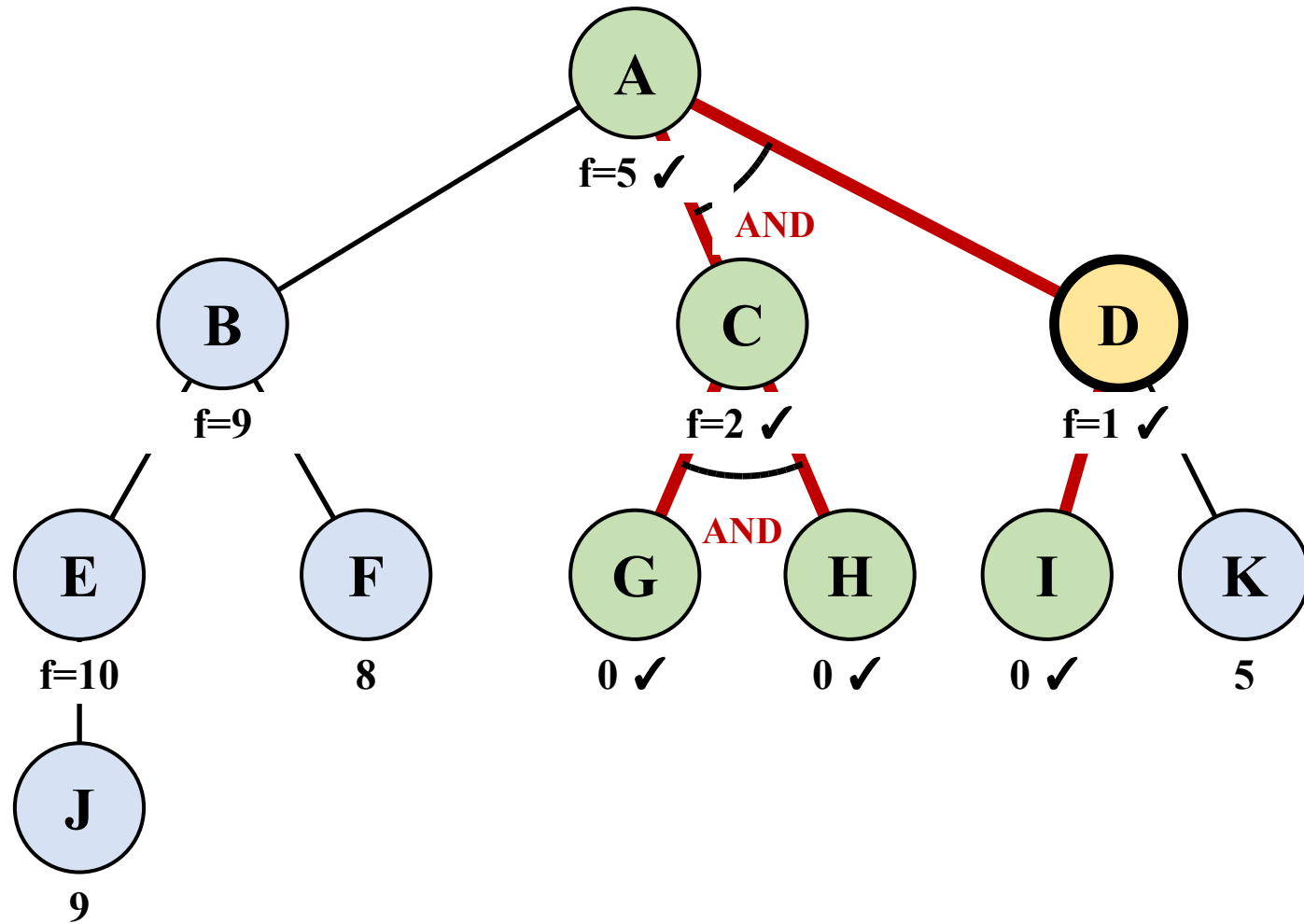


Step 4: expand C

- ▶ $C \rightarrow G \text{ AND } H$ (both primitive: SOLVED, cost 0)
- ▶ $f(C) = (1+0) + (1+0) = 2 \rightarrow C$ SOLVED
- ▶ Revise A: via C&D = $(1+2) + (1+3) = 7$
- ▶ $f(A) = 7$

Expanding now
 Generated
 SOLVED ✓
 best solution so far

AO* Search: Step by Step (5/5)



Step 5: expand D

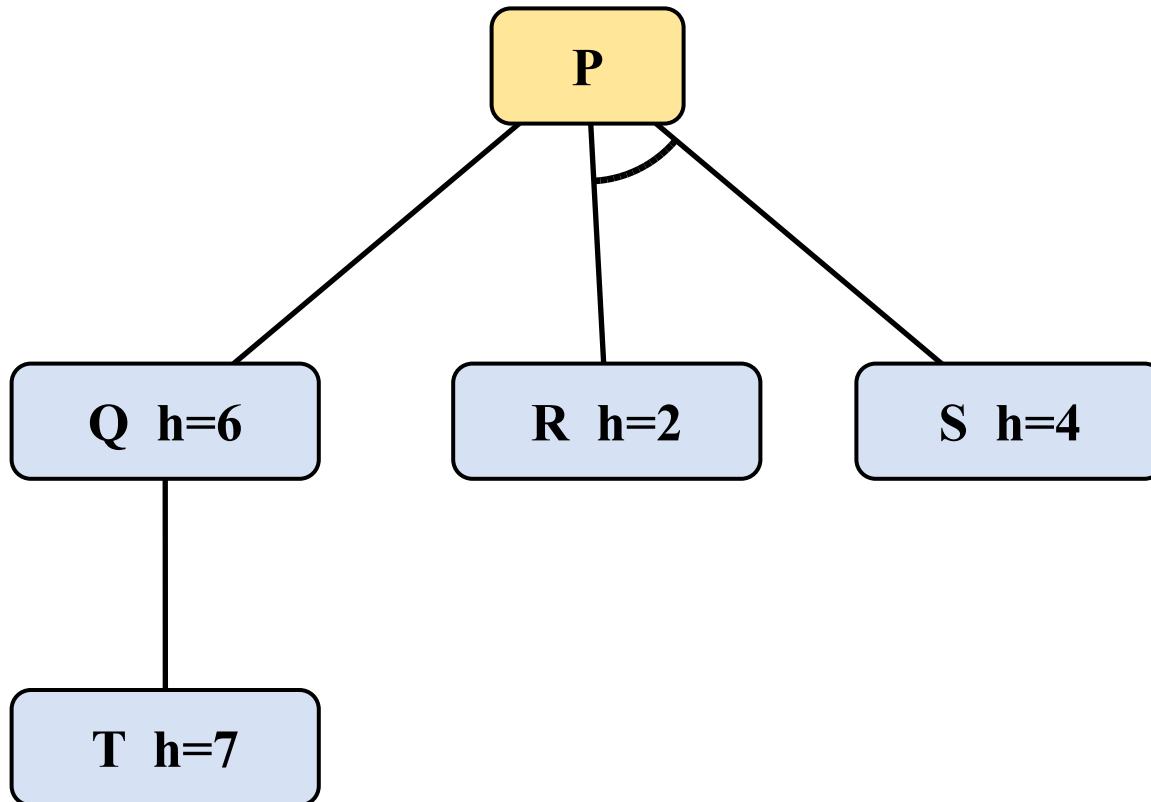
- ▶ $D \rightarrow I$ (SOLVED, 0) OR K (5)
- ▶ $f(D) = \min(1+0, 1+5) = 1 \rightarrow D$ SOLVED
- ▶ $A \text{ via } C\&D = (1+2) + (1+1) = 5$
- ▶ **C and D both SOLVED $\rightarrow$ A SOLVED ✓ Cost = 5**

Expanding now
 Generated
 SOLVED ✓
 best solution so far

A* vs AO*

	A*	AO*
Graph type	OR graph (ordinary state space)	AND–OR graph
Solution	A single path	A solution graph (tree)
Cost	$f(n) = g(n) + h(n)$	OR: minimum; AND: sum of parts
Revises costs?	No (g is fixed once found)	Yes — propagates costs upward
Optimal?	Yes, with admissible h	Yes, with admissible h
Typical use	Route finding, puzzles	Problem decomposition, planning, theorem proving

Class Exercise: AO* (5 minutes)



- ▶ Every edge costs 1.
- ▶ 1. Expand P. Which choice is best? What is $f(P)$?
- ▶ 2. Now expand that node: Q has one child T with $h = 7$.
- ▶ Revise the costs. Does the best choice for P change?

Quick Check

- ▶ 1. Why can hill climbing stop before reaching the goal?
- ▶ 2. Name two ways to escape a local maximum.
- ▶ 3. What is the difference between an AND arc and an OR edge?
- ▶ 4. An AND choice has children with $h = 2$ and $h = 3$ (edge cost 1). What is its cost?
- ▶ 5. Give one difference between A^* and AO^* .

Summary

- ▶ **Local search** keeps one current state; it is fast and needs almost no memory.
- ▶ **Hill climbing** moves to the best neighbour but can get stuck at local maxima, plateaus and ridges.
- ▶ **Random restart** and **simulated annealing** help it escape.
- ▶ **Problem reduction** splits a problem into sub-problems: **OR** = choose one, **AND** = solve all.
- ▶ **AO*** finds the cheapest **solution graph** in an AND–OR graph by expanding the best partial solution and revising costs upward.
- ▶ **Reading: Russell & Norvig 4.1–4.2 (local search, AND–OR search); Rich & Knight, Ch. 3 (problem reduction, AO*).**
- ▶ **Next lecture: Constraint Satisfaction Problems**

Thank You

Any questions?